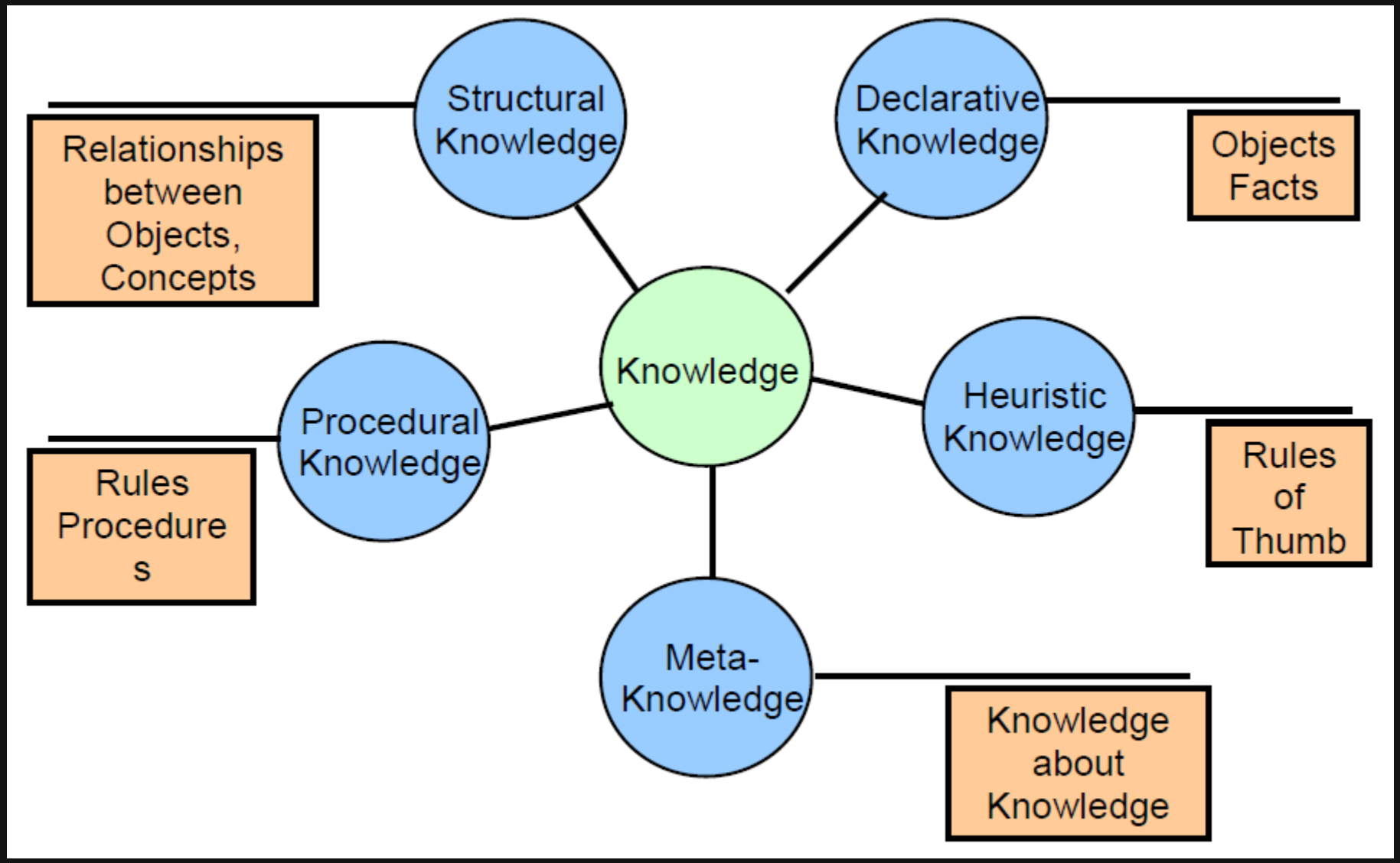
UNIT : 4

Knowledge and Reasoning

Knowledge and Knowledge representation

- Now that have looked at general problem solving, lets look at knowledge representation and reasoning which are important aspects of any artificial intelligence system
- In this section we will become familiar with classical methods of knowledge representation and reasoning in AI.
- Understanding **theoretical or practical aspects of the subject is called as knowledge**
- We can **gain knowledge through experience** acquired based on the **facts**, **information** etc. **about the subject**
- After gaining knowledge about some subject we can apply that knowledge **to derive conclusions** about various problems related to that subject **based on some reasoning**


Knowledge
Structural Knowledge
Relationships between Objects, Concepts
Declarative Knowledge
Objects Facts
Procedural Knowledge
Rules Procedures
Heuristic Knowledge
Rules of Thumb
Meta-Knowledge
Knowledge about Knowledge

A Knowledge Based Agent

- Knowledge base is a **set of representations of facts and information**
- Knowledge level is a **base level(initial knowledge**) of an agent, which **consists of domain-specific content** (information about the surrounding in which the agent is working)
- Knowledge based agent make **use of the existing Knowledge** along **with the current input** from the environment in order **to infer hidden aspects** of the current state
- Agent makes use of **TELL** and **ASK** mechanism

A Knowledge Based Agent

- **TELL(K):** is a function **that adds knowledge 'K' to the knowledge base** (what **needs to know** about surrounding to perform some action)
- **ASK(K):** is a function that **queries the agent about the truth of 'K'** (**what actions** should be carried out to get **desired output**)

Reasoning(Logic)

- Reasoning is associated with thinking, cognition, and intellect.
- The philosophical field of logical studies in which humans reason formally through arguments
- Reasoning may be subdivided into many forms, we are going to study only the following -
  - ✓ Propositional Logic(PL)
  - ✓ First Order Predicate Logic

Propositional Logic(PL)

- PL is a simple knowledge representation language
- A proposition **is the statement of a fact**.
- We usually assign a symbolic variable (Symbols are denoted by capital letters like A,B,C, etc.) to represent a proposition, e.g.
  - P = It is raining
  - Q = I carry an umbrella
- A proposition is a sentence whose truth values may be determined. So, each proposition has a truth value,(Constants have truth values i.e. 0 is False and 1 is True) e.g.
  - The proposition 'A rectangle has four sides' is true
  - The proposition 'The world is a cube' is false.
- PL is the foundation for higher logics like First Order Logic(FOL)

Propositional Logic(PL)

- Different propositions may be logically related and we can form compound statements of propositions using logical connectives.
- Common logical connectives are:

1. ∧ AND (Conjunction)
2. ∨ OR (Disjunction)
3. ¬ NOT (Negation)
4. => If … then (Conditional)
5. <=> If and only if (bi-conditional)

Propositional Logic(PL)

Connectives used in Propositional logic

Connective symbol	Name of the Connective symbol	Relationship between Propositional symbols	Name of the Relationship between Propositional symbols
$\wedge$	And	$A \wedge B$	Conjunction
$\vee$	Or	$A \vee B$	Disjunction
$\neg$	Not	$\neg A$	Negation
$\Rightarrow$	Implies	$A \Rightarrow B$	Implication / conditional
$\Leftrightarrow$	is equivalent/ if and only if	$A \Leftrightarrow B$	Biconditional

Propositional Logic(PL)

- To define logical connectives truth tables are used
- Following Truth Table shows five connectives

A	B	A∧B	A∨B	¬A	A=>B	A<=>B
T	T					
T	F					
F	T					
F	F					

Propositional Logic(PL)

- To define logical connectives truth tables are used
- Following Truth Table shows five connectives

A	B	A∧B	A∨B	¬A	A=>B	A<=>B
T	T	T				
T	F	F				
F	T	F				
F	F	F				

Propositional Logic(PL)

- To define logical connectives truth tables are used
- Following Truth Table shows five connectives

A	B	A∧B	A∨B	¬A	A=>B	A<=>B
T	T	T	T			
T	F	F	T			
F	T	F	T			
F	F	F	F			

Propositional Logic(PL)

- To define logical connectives truth tables are used
- Following Truth Table shows five connectives

A	B	A∧B	A∨B	¬A	A=>B	A<=>B
T	T	T	T	F		
T	F	F	T	F		
F	T	F	T	T		
F	F	F	F	T		

Propositional Logic(PL)

- To define logical connectives truth tables are used
- A=>B equal to ¬A ∨ B

A	B	A∧B	A∨B	¬A	A=>B	A<=>B
T	T	T	T	F	T	
T	F	F	T	F	F	
F	T	F	T	T	T	
F	F	F	F	T	T	

Propositional Logic(PL)

- To define logical connectives truth tables are used
- A<=>B equal to (A=>B) ∧ (B=>A)

A	B	A∧B	A∨B	¬A	A=>B	A<=>B
T	T	T	T	F	T	T
T	F	F	T	F	F	F
F	T	F	T	T	T	F
F	F	F	F	T	T	T

Propositional Logic(PL)

Conversion of sentences into PL

1. It is hot and sunny
   - A: it is hot
   - B: it is sunny
   - ✓ $A \wedge B$

2. She can have either tea or coffee
   - A: She can have tea
   - B: she can have coffee
   - ✓ $A \vee B$

Propositional Logic(PL)

Conversion of sentences into PL

3. I don't like Stephen but I like Peter

- A: I like Stephen
- B: I like Peter
- ✓ ¬ A ∧ B

4. If it is humid then it will rain

- A: it is humid
- B: it will rain
- ✓ A => B

Propositional Logic(PL)

Conversion of sentences into PL

5. John is good looking and wealthy but not healthy

- A: John is good looking
- B: John is wealthy
- C: John is healthy

✓ $(A \wedge B) \wedge \neg C$

6. Ram is the English teacher but not Maths teacher

- A: Ram is a English teacher
- B: Ram is a Maths teacher

✓ $A \wedge \neg B$

Propositional Logic(PL)

Conversion of sentences into PL

7. If it is rainy I will stay at home

- A: it is rainy
- B: I will stay at home
- A => B

8. If John brought an i-phone today then he either sold his old mobile or took a bank loan

- A: John brought i-phone
- B: John sold his old mobile
- C: John took a bank loan
- ✓ A => (B ∨ C)

Propositional Logic(PL)

Conversion of sentences into PL

9. If it is hot and humid then it is raining

- A: it is hot
- B: it is humid
- C: it is raining

✓ $(A \wedge B) \Rightarrow C$

10. Cat chases mice or bird but not at a same time

- A: cat chases mice
- B: cat chases bird

✓ $(A \vee B) \wedge \neg (A \wedge B)$

Propositional Logic(PL)

Conversion of sentences into PL

11. Dolphin's neither literate not able to read but they are intelligent
    - A: Dolphin's are literate
    - B: Dolphin's are able to read
    - C: Dolphin's are intelligent
    - ✓ $(\neg A \vee \neg B) \wedge C$

Propositional Logic(PL)

PROPOSITIONAL THEOREM

$$
\begin{aligned}
(\alpha \wedge \beta) &\equiv (\beta \wedge \alpha) && \text{commutativity of } \wedge \\
(\alpha \vee \beta) &\equiv (\beta \vee \alpha) && \text{commutativity of } \vee \\
((\alpha \wedge \beta) \wedge \gamma) &\equiv (\alpha \wedge (\beta \wedge \gamma)) && \text{associativity of } \wedge \\
((\alpha \vee \beta) \vee \gamma) &\equiv (\alpha \vee (\beta \vee \gamma)) && \text{associativity of } \vee \\
\neg(\neg\alpha) &\equiv \alpha && \text{double-negation elimination} \\
(\alpha \Rightarrow \beta) &\equiv (\neg\beta \Rightarrow \neg\alpha) && \text{contraposition} \\
(\alpha \Rightarrow \beta) &\equiv (\neg\alpha \vee \beta) && \text{implication elimination} \\
(\alpha \Leftrightarrow \beta) &\equiv ((\alpha \Rightarrow \beta) \wedge (\beta \Rightarrow \alpha)) && \text{biconditional elimination} \\
\neg(\alpha \wedge \beta) &\equiv (\neg\alpha \vee \neg\beta) && \text{De Morgan} \\
\neg(\alpha \vee \beta) &\equiv (\neg\alpha \wedge \neg\beta) && \text{De Morgan} \\
(\alpha \wedge (\beta \vee \gamma)) &\equiv ((\alpha \wedge \beta) \vee (\alpha \wedge \gamma)) && \text{distributivity of } \wedge \text{ over } \vee \\
(\alpha \vee (\beta \wedge \gamma)) &\equiv ((\alpha \vee \beta) \wedge (\alpha \vee \gamma)) && \text{distributivity of } \vee \text{ over } \wedge
\end{aligned}
$$

Standard logical equivalences. The symbols α, β, and γ stand for arbitrary sentences of propositional logic.

Conjunctive Normal Form

Propositional logic, a formula is in Conjunctive Normal Form (CNF) if it is a conjunction (AND, ∧) of one or more clauses, where each clause is a disjunction (OR, ∨) of literal's.
A literal is either a proposition (e.g., P) or its negation (e.g., ¬P).

Example of CNF:

$(A \vee B) \wedge (\neg C \vee D) \wedge (E$

Propositional Logic(PL)

Conversion from PL to CNF(Conjunctive Normal Form)

(e) Covert the following propositional logic statement into CNF

(i) $A \rightarrow (B \leftarrow\rightarrow C)$

Q2. $(A \rightarrow B) \rightarrow C$

Q.3 $(A \vee B) \rightarrow (C \rightarrow D)$

Q.4 $\neg ((P \vee \neg Q) \rightarrow R) \rightarrow (P \wedge R))$

Convert $(A \to (B \leftrightarrow C))$ to CNF:

Step 1: Eliminate Biconditional ($\leftrightarrow$)

$$B \leftrightarrow C \equiv (\neg B \vee C) \wedge (\neg C \vee B)$$

Rewrite with Implication:

$$A \to ((\neg B \vee C) \wedge (\neg C \vee B))$$

Step 2: Eliminate Implication ($\to$)

$$A \to X \equiv \neg A \vee X$$

$$\equiv \neg A \vee ((\neg B \vee C) \wedge (\neg C \vee B))$$

Step 3: Distribute OR over AND:

$$\equiv (\neg A \vee \neg B \vee C) \wedge (\neg A \vee \neg C \vee B)$$

Q2: $(A \to B) \to C$

Step 1: Eliminate Implications:

$$(A \to B) \to C \equiv (\neg A \vee B) \to C$$

$$\equiv \neg(\neg A \vee B) \vee C$$

Step 2: Apply De Morgan's Law:

$$\equiv (A \wedge \neg B) \vee C$$

Step 3: Distribute OR over AND:

$$\equiv (A \vee C) \wedge (\neg B \vee C)$$

Q3: $(A \vee B) \rightarrow (C \rightarrow D)$

Step 1: Eliminate Implications:

$$(A \vee B) \rightarrow (C \rightarrow D) \equiv \neg(A \vee B) \vee (\neg C \vee D)$$

Step 2: Apply De Morgan's Law:

$$\equiv (\neg A \wedge \neg B) \vee (\neg C \vee D)$$

Step 3: Distribute OR over AND:

$$\equiv (\neg A \vee \neg C \vee D) \wedge (\neg B \vee \neg C \vee D)$$

CNF Form:

$$(\neg A \vee \neg C \vee D) \wedge (\neg B \vee \neg C \vee D)$$

Q4: $\neg((P \vee \neg Q) \rightarrow R) \rightarrow (P \wedge R)$

Step 1: Eliminate Implications:

$$\neg((P \vee \neg Q) \rightarrow R) \rightarrow (P \wedge R)$$

$$\equiv \neg(\neg(P \vee \neg Q) \vee R) \rightarrow (P \wedge R)$$

$$\equiv \neg(\neg(P \vee \neg Q) \vee R) \vee (P \wedge R)$$

Step 2: Apply De Morgan's Law:

$$\equiv ((P \vee \neg Q) \wedge \neg R) \vee (P \wedge R)$$

Step 3: Distribute OR over AND:

$$\equiv ((P \vee \neg Q \vee P) \wedge (P \vee \neg Q \vee R) \wedge (\neg R \vee P) \wedge (\neg R \vee R))$$

Resolution Tree

If fact ‘F’ is to be proved then assume ‘¬F’ and start

It contradict all the other rules in knowledge base

The process stops, when it returns ‘NULL’

Prove that “Is someone smiling?” using resolution

So, start with what you want to prove with negative value

¬ Smile(W)

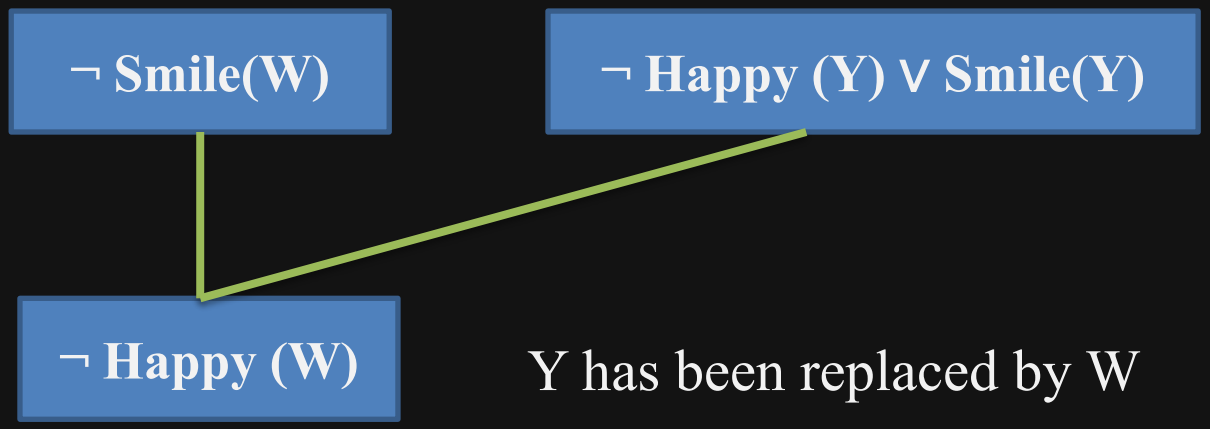
Now search into the list of rules for smile (i.e. Fact 2) and draw it in front of first box, like

¬ Smile(W)

¬ Happy (Y) ∨ Smile(Y)

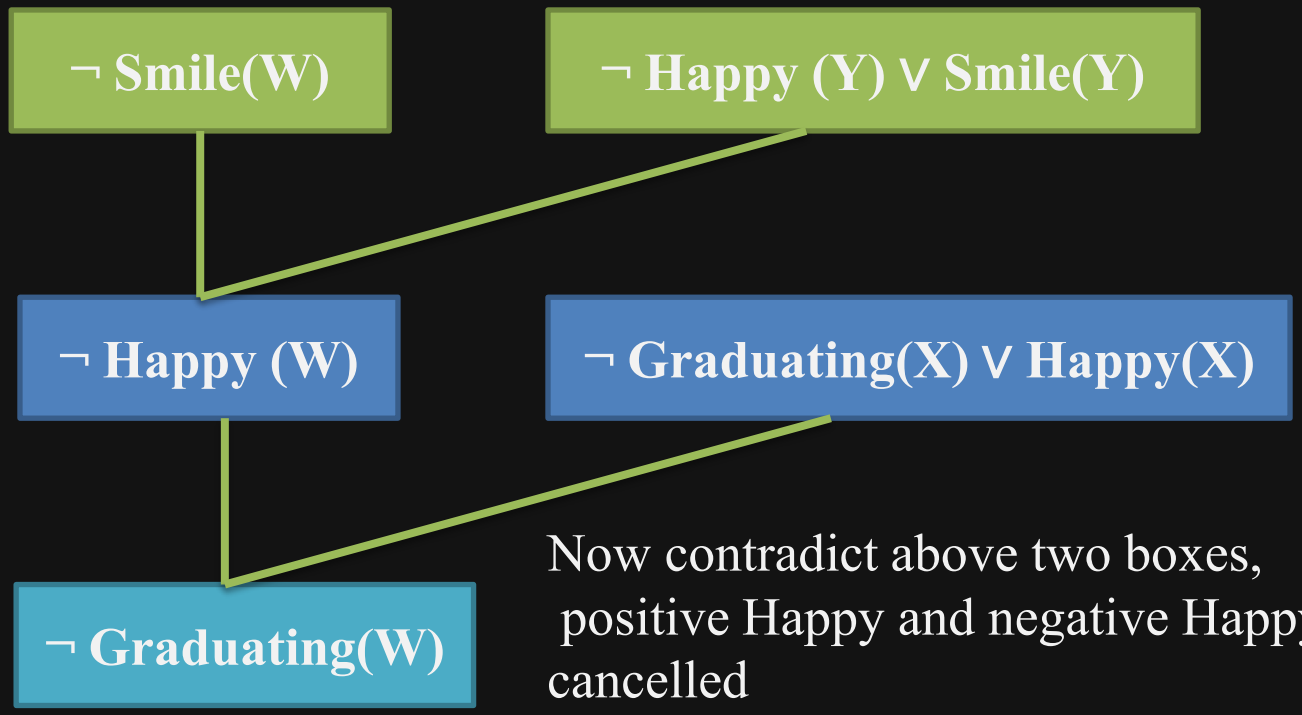
Resolution Tree

Now contradict/compare above two boxes, positive smile and negative smile get cancelled



Resolution Tree

Now find/search happy from the list of facts (i.e. fact 1)

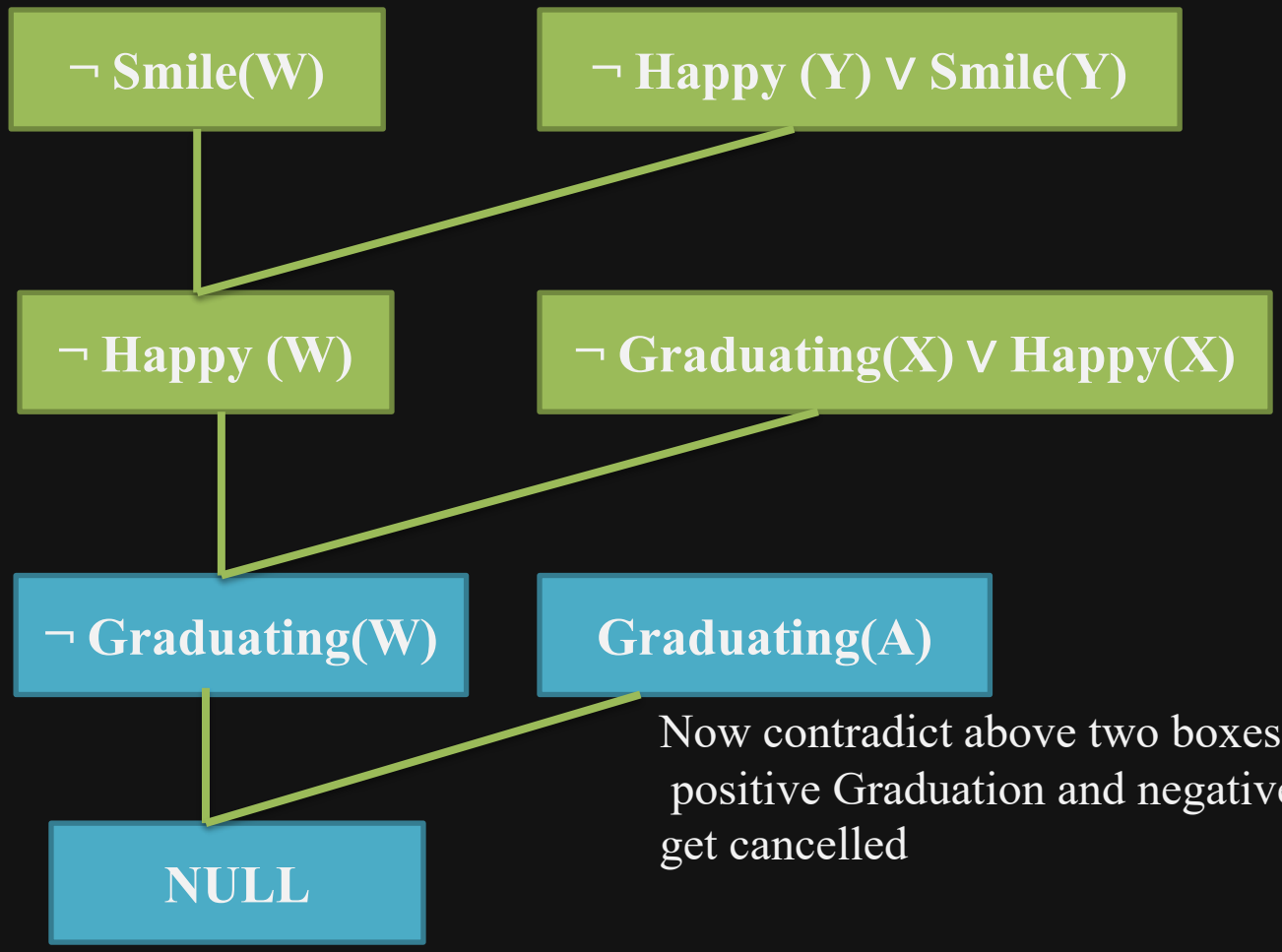


Now contradict above two boxes, positive Happy and negative Happy get cancelled

X has been replaced by W

Resolution Tree

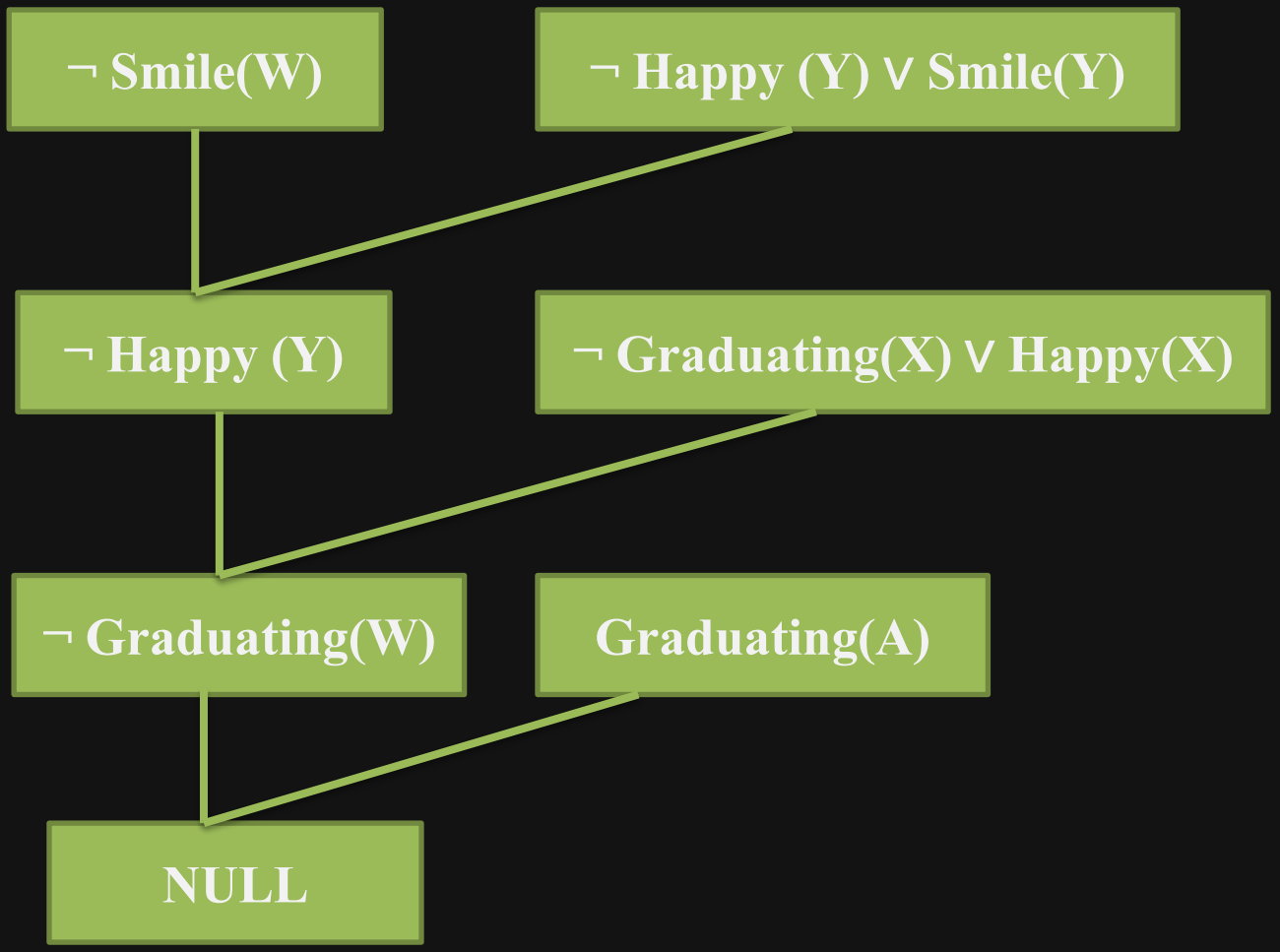
Now find/search Graduation from the list of facts (i.e. fact 3)



Now contradict above two boxes,
positive Graduation and negative Graduation
get cancelled

Final Resolution Tree

Hence “Someone is not smiling” is Wrong, so “Someone is smiling” is Proved



UNIT-5 Planning

Introduction to Planning

- Lets take an example.
- Going on a Trip
- Book tickets
- Reserve a hotel
- Pack luggage
- Arrange transport to the airport
- One thing easily visible is that this problem can be broken into multiple problems
- i.e. is composed of small problems like packing luggage and finding the hotel.
- Another observation is that different things are dependent on others. Dependencies: Packing depends on trip duration (which depends on booking)Airport travel depends on flight timing

Introduction to Planning

- Planning in AI can be defined as a problem that needs decision making by intelligent systems to accomplish the given task/target
- The intelligent system may be a robot or computer program
- Planning is an activity where agent has to come up with a sequence of actions to accomplish a task/target
- The aim of an agent is to find the proper sequence of action which will lead from starting state to goal state and produce an efficient solution.

Planning

A classical planning has the following assumptions about the task environment

1. Fully observable
2. Deterministic: agent can determine the consequences of its actions
3. Finite
4. Static
5. Discrete:A discrete system is one where things happen in separate, distinct steps, not continuously.

So basically a planning problem finds the sequence of action to accomplish the goal, based on the above assumptions.

Planning

- In planning, goal can be specified as a union of sub-goals
- Advantage of planning is that, the order of planning and the order of execution need not be same.
- e.g. You can plan how to pay bills of grocery before planning to go to supermarket
- Planning make use of divide and conquer policy by dividing the goal into sub-goals

Planning with state-space search

- Suppose an agent that can perform three tasks that are -printing, sending a mail and making coffee (let's call agent as office agent)
- When this office agent gets orders from three people at a same time to perform these three tasks
- An agent has to decide which task can be performed more efficiently in lessor time
- If some input and output locations are closer on the state-space grid, then the probability of performing the task increases.

Block World Problem

- One of the most famous planning domain is known as the blocks world.
- This domain consists of a set of cube-shaped blocks sitting on a table.

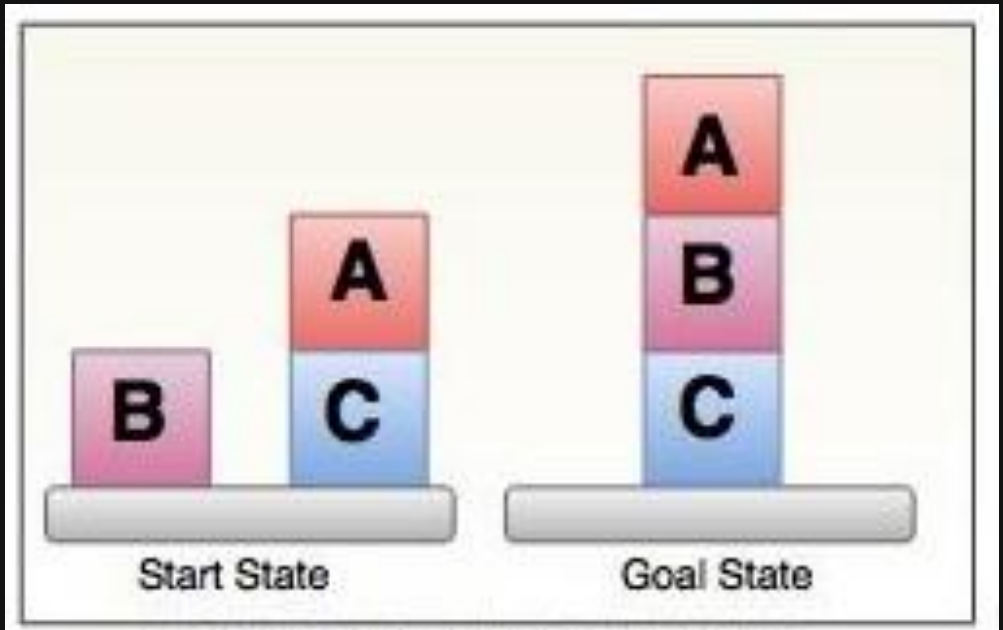


Fig: Blocks-World Planning Problem

Block World Problem

- The blocks can be stacked, but only one block can fit directly on top of another. A robot arm can pick up a block and move it to another position, either on the table or on top of another block.
- The arm can pick up only one block at a time, so it cannot pick up a block that has another one on it.
- The goal will always be to build one or more stacks of blocks, specified in terms of what blocks are on top of what other blocks.

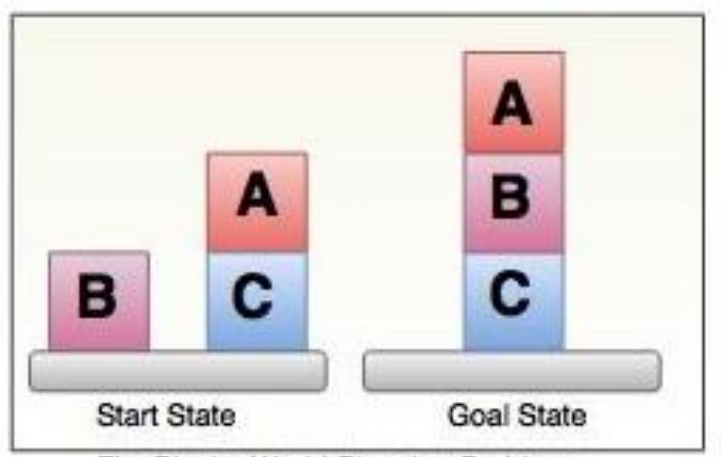


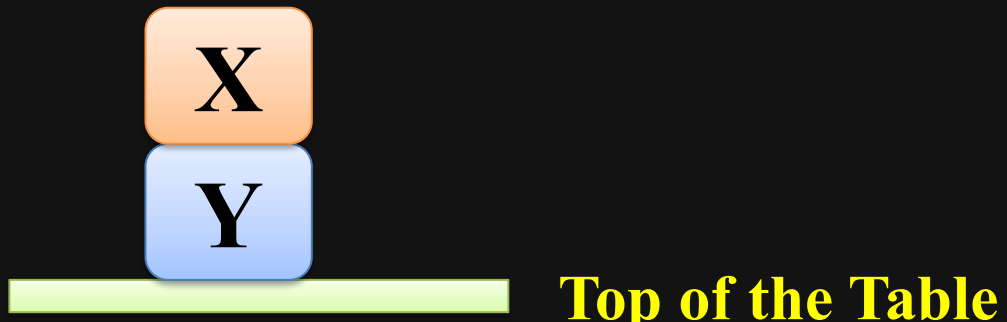
Fig: Blocks-World Planning Problem

Block World Problem

Actions to perform:

Action 1: **on(X,Y)**

- To indicate that Block ‘X’ is on Block ‘Y’
- Where ‘Y’ is either another block or on the top of a Table

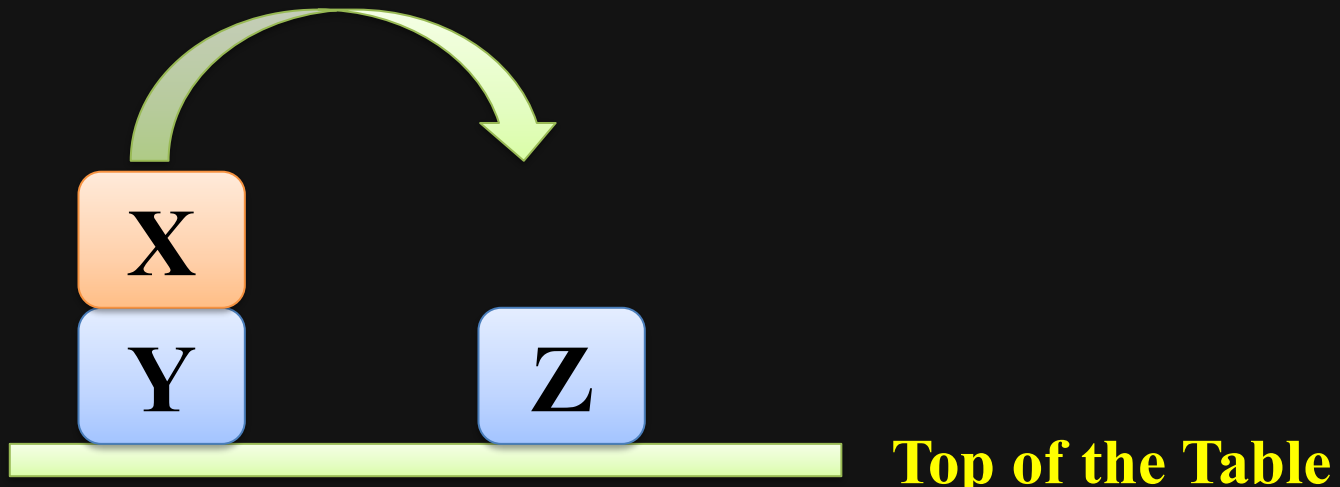


Block World Problem

Action 2: **move(X,Y,Z)**

- To move Block ‘X’ from top of ‘Y’ to top of ‘Z’
- Preconditions: no other Block on ‘X’ i.e. clear(X)

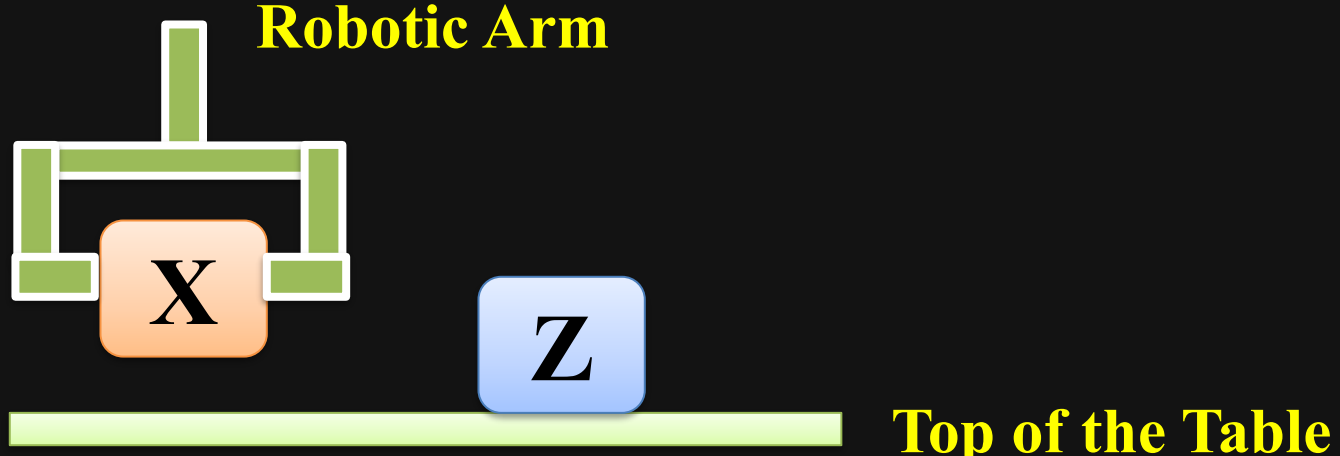
Clear(x) is true when nothing is on x.



Block World Problem

Action 3: **pickup(X)**

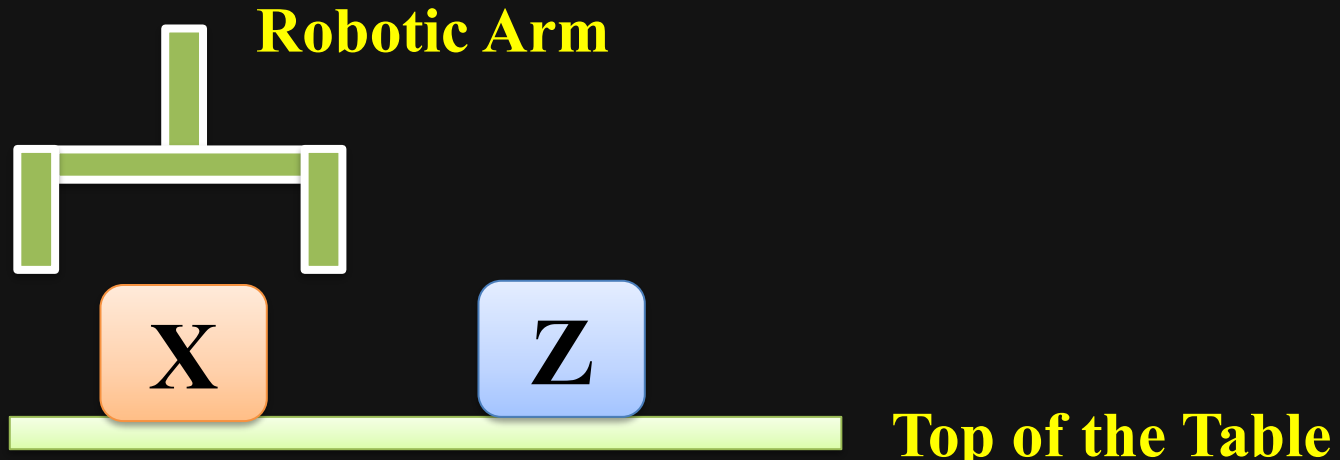
- To pickup Block ‘X’ from top of any block or from the top of Table
- Preconditions: Arm Empty, clear(X)



Block World Problem

Action 4: **putdown(X)**

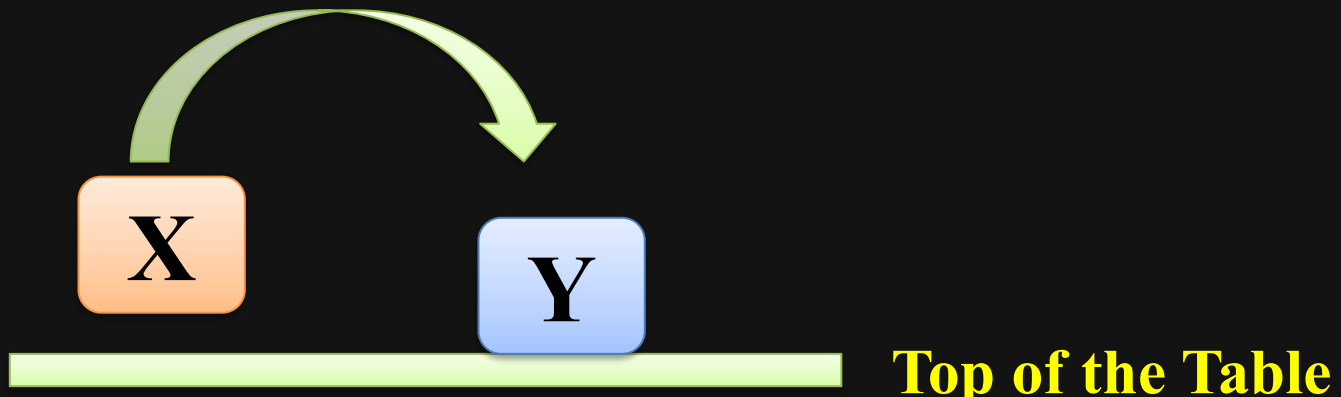
- To put down Block ‘X’ on top of any block or on the top of Table
- Preconditions: hold(X) i.e. Arm should holding Block ‘X’



Block World Problem

Action 5: **stack(X,Y)**

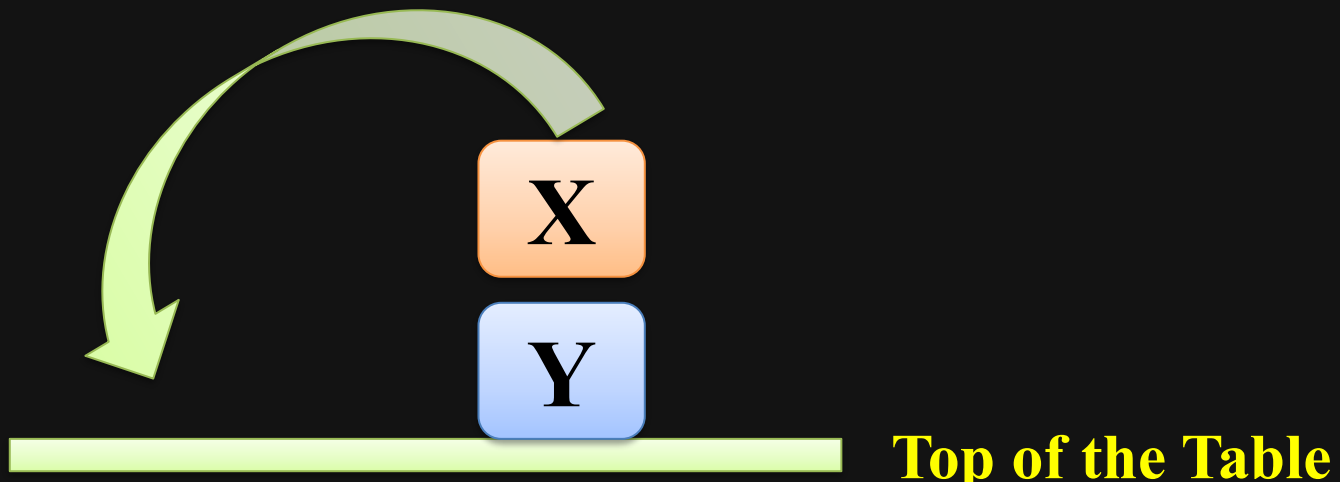
- To put Block ‘X’ on to the Block ‘Y’
- Preconditions: hold(X) and clear(Y)



Block World Problem

Action 6: **unstack(X,Y)**

- To remove Block ‘X’ from the top of Block ‘Y’
- Preconditions: Arm Empty, on(X,Y) and clear(X)



Block World Problem

Planning to solve Block world problem of following diagram

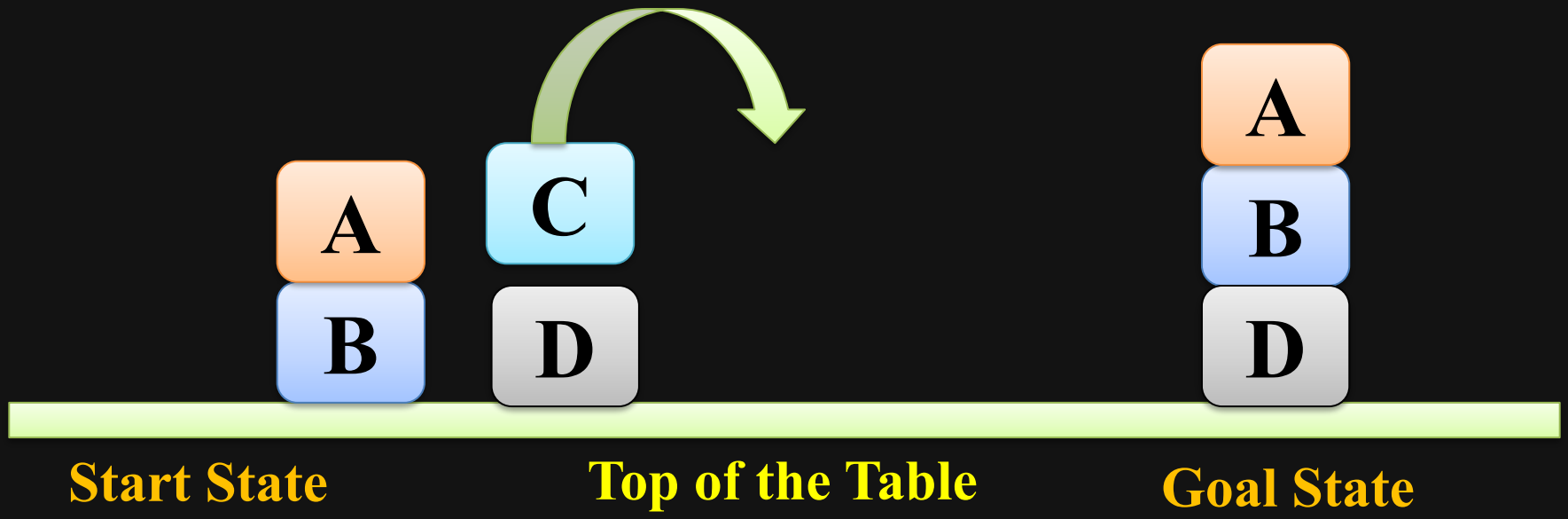
The goal is to get Block ‘A’ on ‘B’ and Block ‘B’ on ‘D’

Block World Problem

Action 1: unstack(C,D)

unstack(C,D)

- To remove Block ‘C’ from the top of Block ‘D’
- Preconditions: Arm Empty, on(C,D) and clear(C)

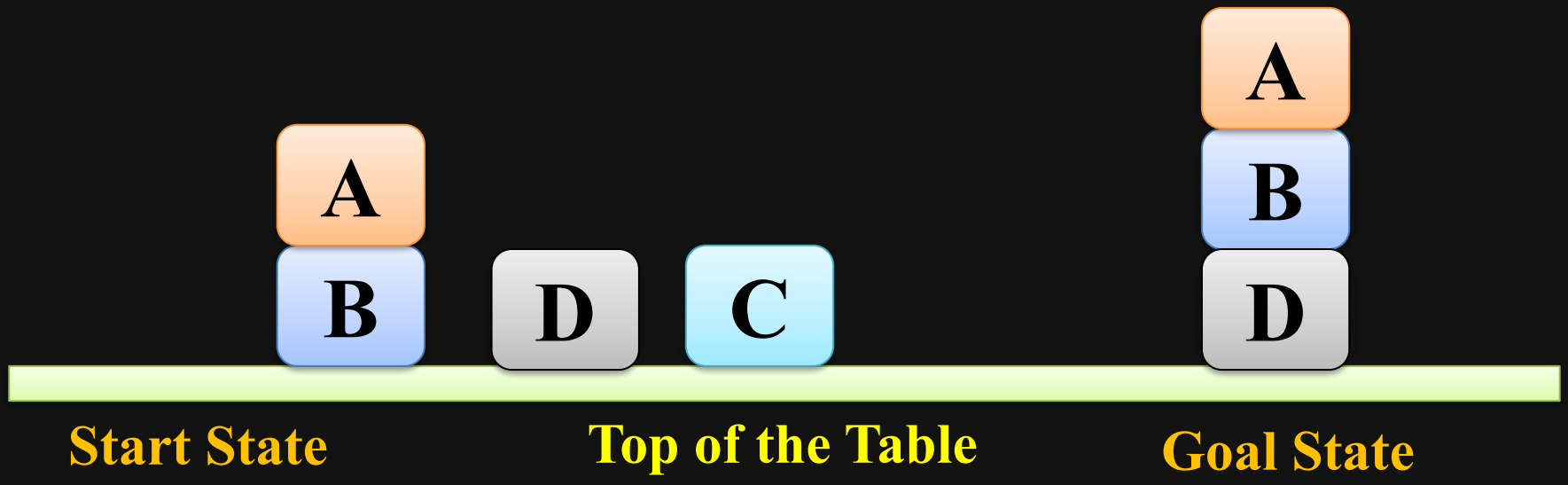


Block World Problem

Action 2: putdown(C)

putdown(C)

- To put down Block ‘C’ on top of any block or on the top of Table
- Preconditions: hold(C) i.e. Arm should holding Block ‘C’

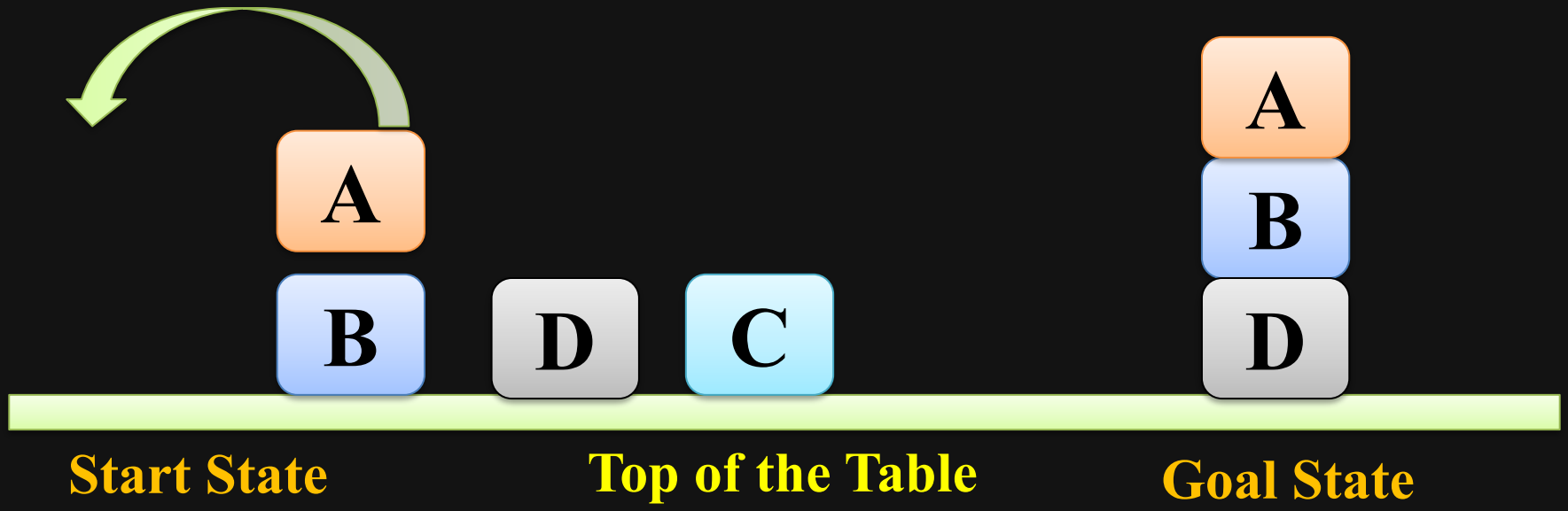


Block World Problem

Action 3: unstack(A,B)

unstack(A,B)

- To remove Block ‘A’ from the top of Block ‘B’
- Preconditions: Arm Empty, on(A,B) and clear(A)

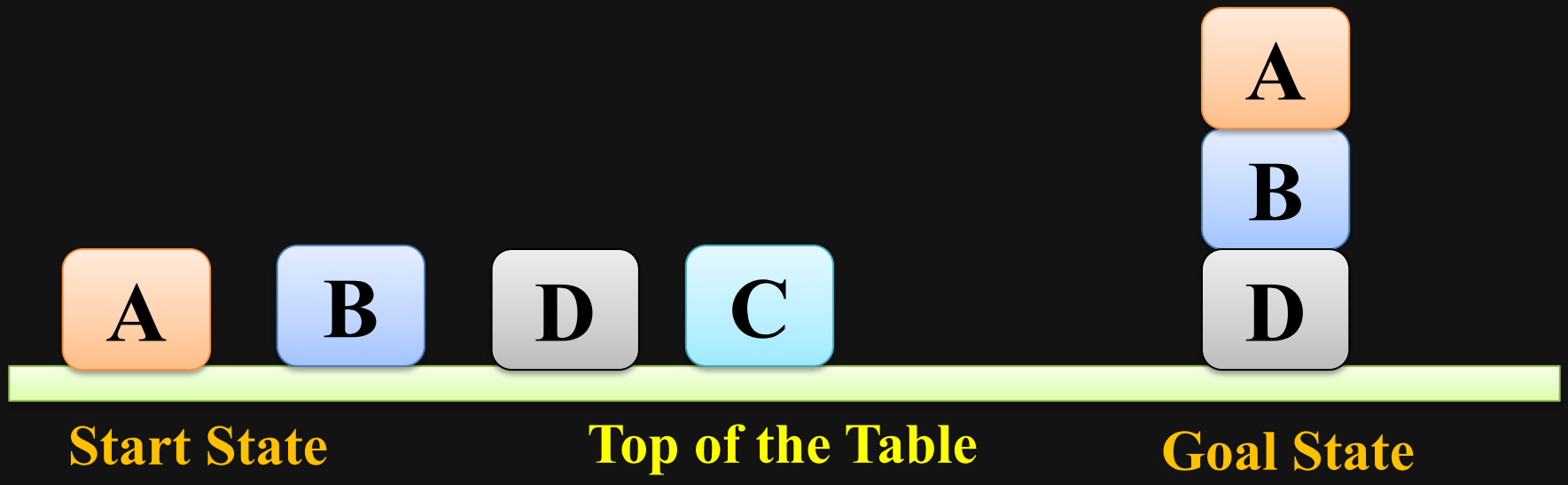


Block World Problem

Action 4: putdown(A)

putdown(A)

- To put down Block ‘A’ on top of any block or on the top of Table
- Preconditions: hold(A) i.e. Arm should holding Block ‘A’

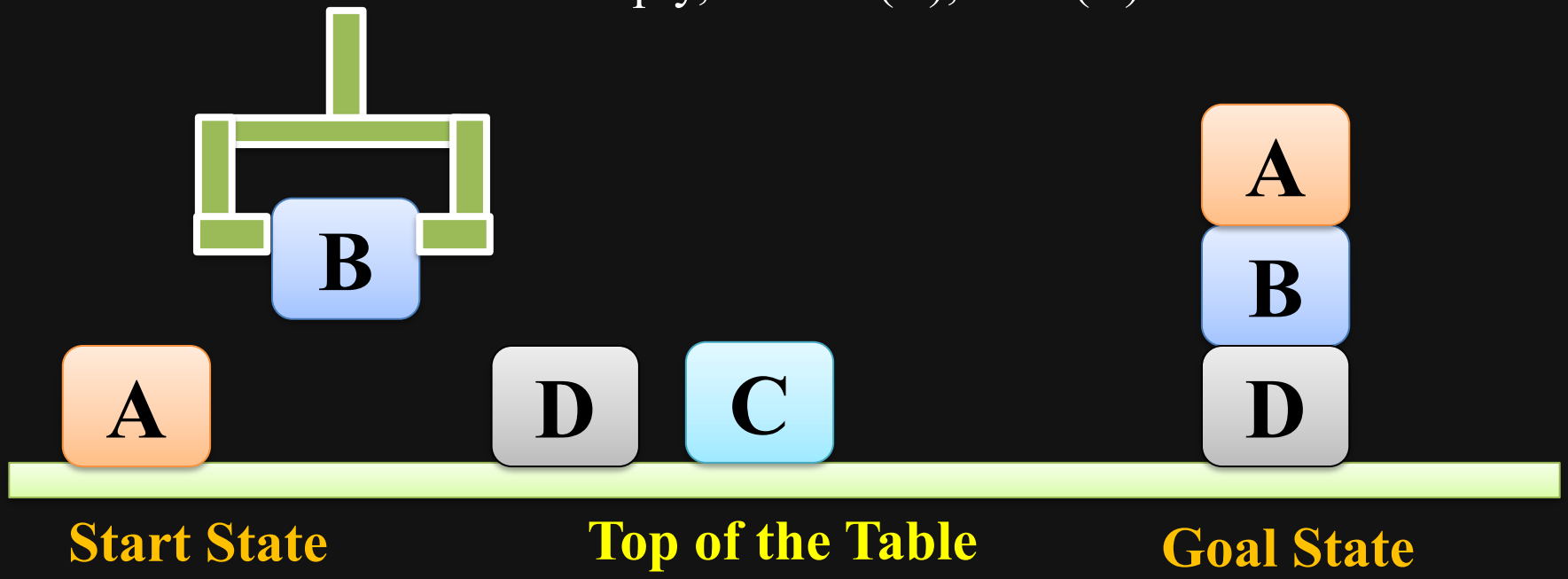


Block World Problem

Action 5: pickup(B)

pickup(B) : To pickup Block ‘B’ from top of any block or from the top of Table

- Preconditions: Arm Empty, ontable(B), clear(B)

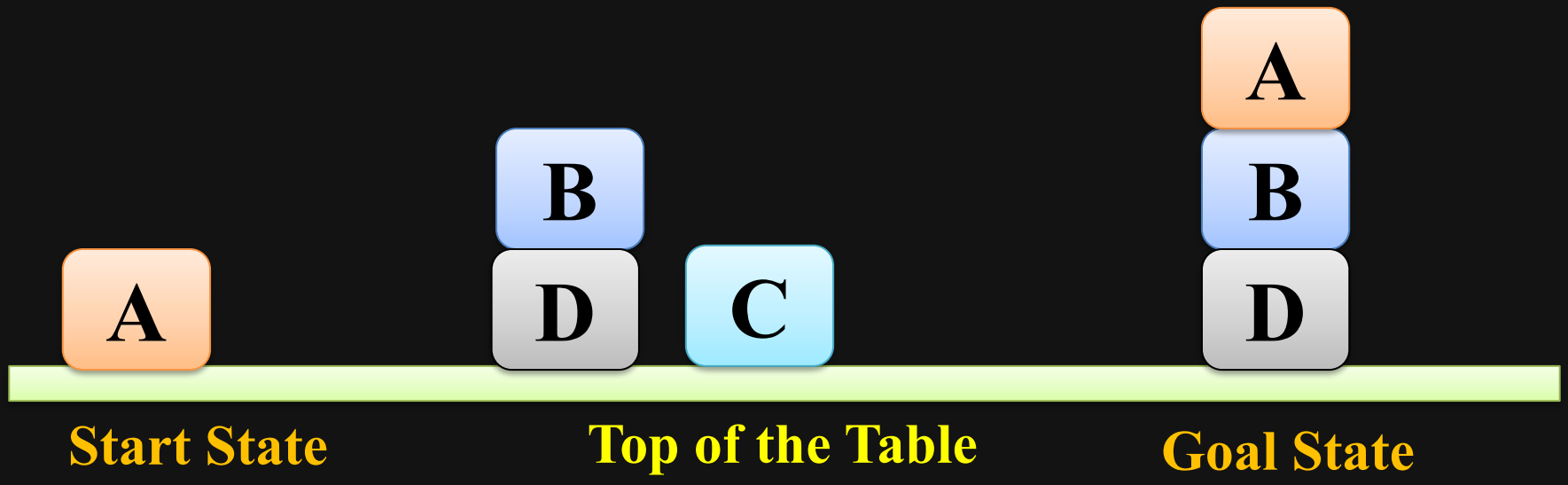


Block World Problem

Action 6: stack(B,D)

stack(B,D)

- To put Block ‘B’ on to the Block ‘D’
- Preconditions: hold(B) and clear(D)

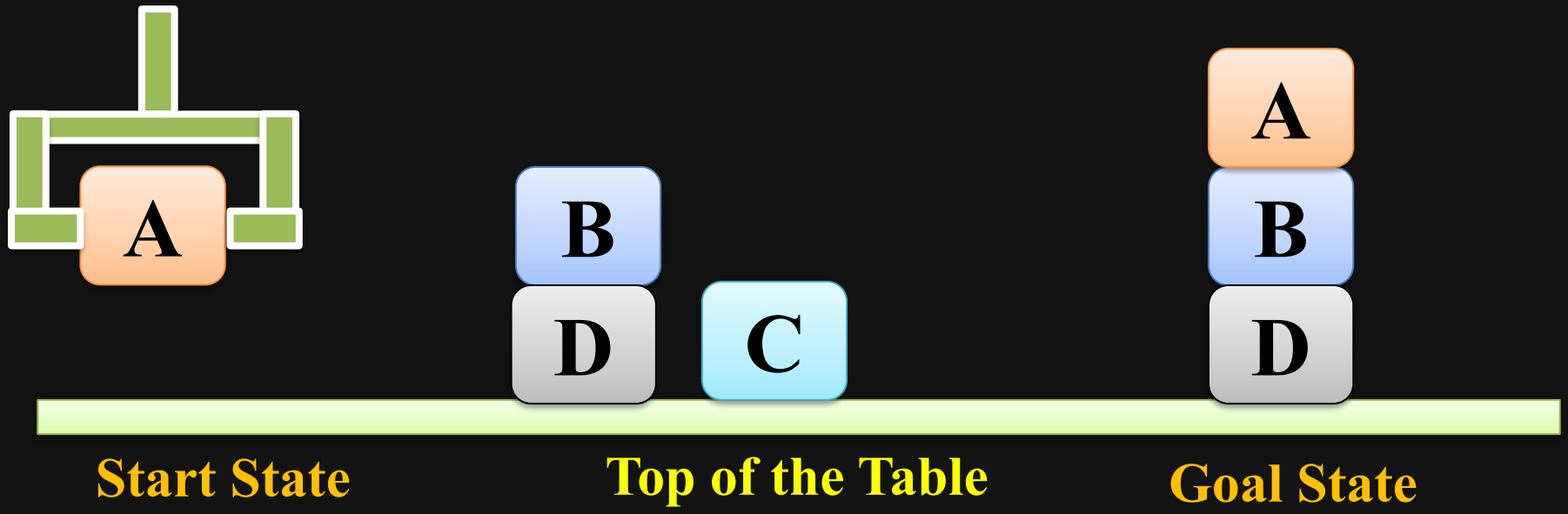


Block World Problem

Action 7: pickup(A)

pickup(A) : To pickup Block ‘A’ from top of any block or from the top of Table

- Preconditions: Arm Empty, ontable(A), clear(A)

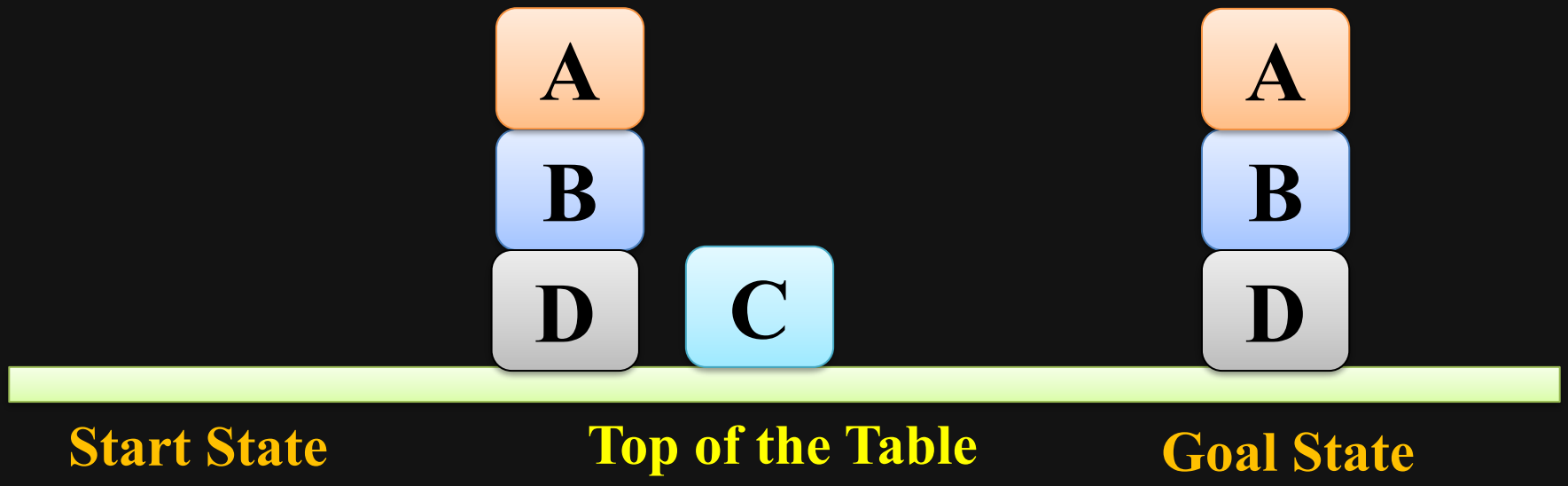


Block World Problem

Action 8: stack(A,B)

stack(A,B)

- To put Block ‘A’ on to the Block ‘B’
- Preconditions: hold(A) and clear(B)



Block World Problem

Planning to solve Block world problem of following diagram

Block World Problem

Planning to solve Block world problem of following diagram

The goal is to get Block ‘A’ on ‘B’ and Block ‘B’ on ‘C’

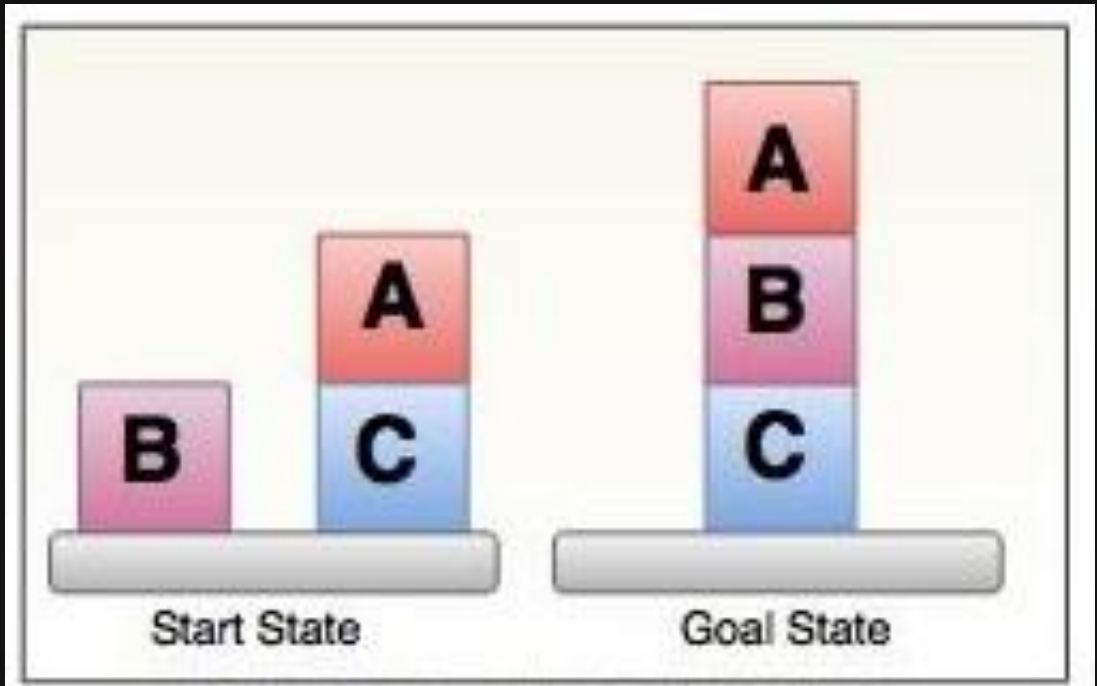


Fig: Blocks-World Planning Problem

Block World Problem

Exercise:

Consider a state S = {(on A B), (on B C), (clear A), ArmEmpty} and an action a = (unstack B C). The preconditions of action a are {(on B C), (clear B), ArmEmpty} and its effects are {(holding B), (clearC), ¬ArmEmpty}. What would be the new state S' generated when we consider this action a for the state S.

a) S' = {(on A B), (holding B), (clear C), ¬ArmEmpty }

b) S' = {(on A B), (holding B), (clear A), (clear C), ¬ArmEmpty }

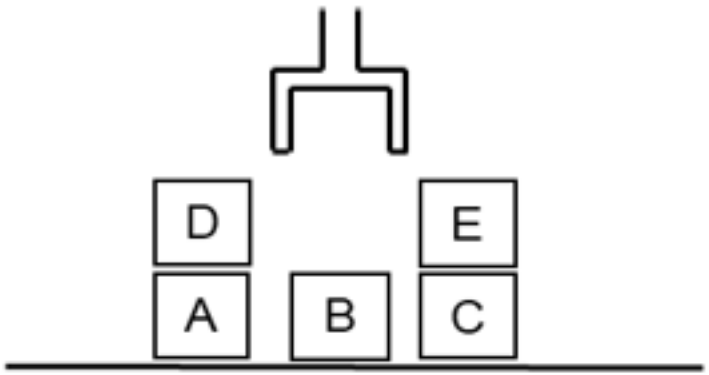
c) The action a is not applicable in state S

d) None of the above

ANS: C because the precondition of action a clear(B) is not met in S

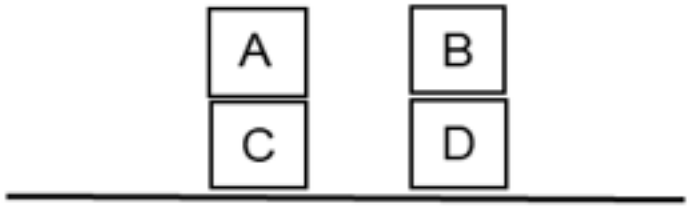
Start

onTable(A), onTable(B), onTable(C),
on(D,A), on(E,C)
clear(D), clear(B), clear(E), AE



Goal

on(B,D), on(A,C)



Q1. Which of the following are applicable actions for the above problem?

a) Pickup(A)

b) Pickup(B)

c) Pickup(E)

d) Unstack(D,A)

e) Unstack(B,D)

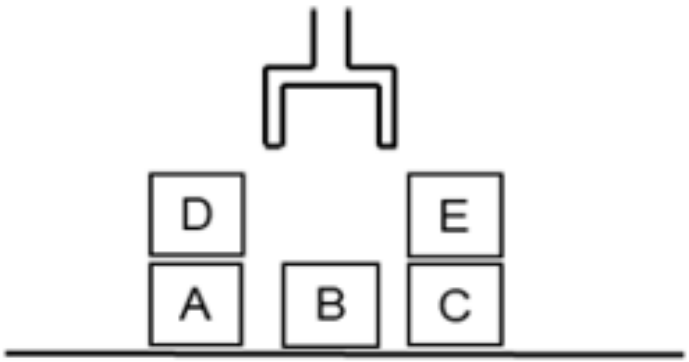
f) Unstack(A,C)

Hint : applicable actions are those whose preconditions are true in the initial state.

Ans: b & d

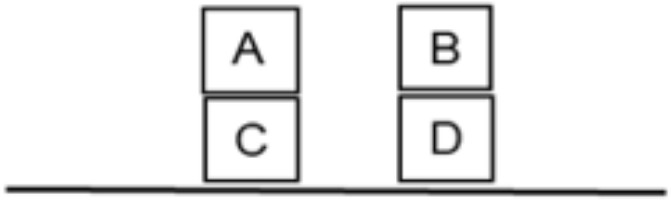
Start

onTable(A), onTable(B), onTable(C),
on(D,A), on(E,C)
clear(D), clear(B), clear(E), AE



Goal

on(B,D), on(A,C)



Q1. Which of the following are applicable actions for the above problem?

a) Pickup(A)

b) Pickup(B)

c) Pickup(E)

d) Unstack(D,A)

e) Unstack(B,D)

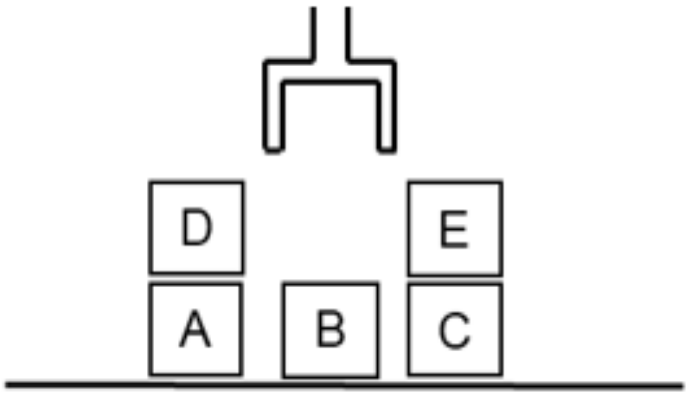
f) Unstack(A,C)

Hint : applicable actions are those whose preconditions are true in the initial state.

Ans: b & d

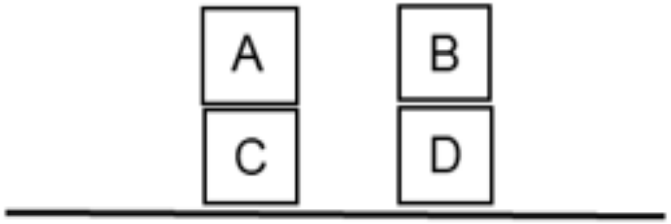
Start

onTable(A), onTable(B), onTable(C),
on(D,A), on(E,C)
clear(D), clear(B), clear(E), AE



Goal

on(B,D), on(A,C)



Q2. Let State Space Planning find a shortest plan. Which of the following can be the first action of the shortest plan?

a) Putdown(C)

b) Pickup(B)

c) Pickup(E)

d) Unstack(D,A)

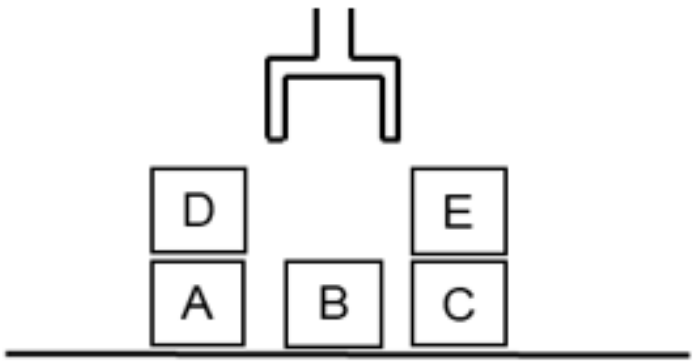
e) Unstack(B,D)

f) Unstack(E,C)

Ans : d & f

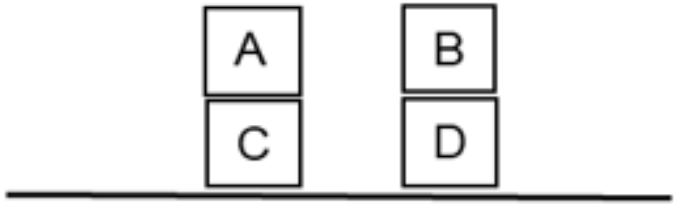
Start

onTable(A), onTable(B), onTable(C),
on(D,A), on(E,C)
clear(D), clear(B), clear(E), AE



Goal

on(B,D), on(A,C)



Q2. Let State Space Planning find a shortest plan. Which of the following can be the first action of the shortest plan?

a) Putdown(C)

b) Pickup(B)

c) Pickup(E)

d) Unstack(D,A)

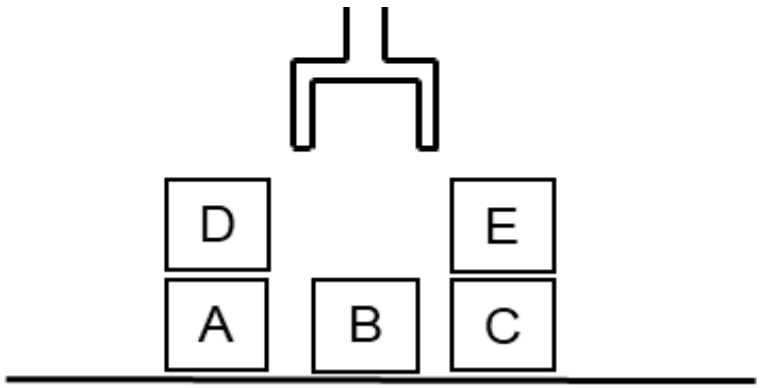
e) Unstack(B,D)

f) Unstack(E,C)

Ans : d & f

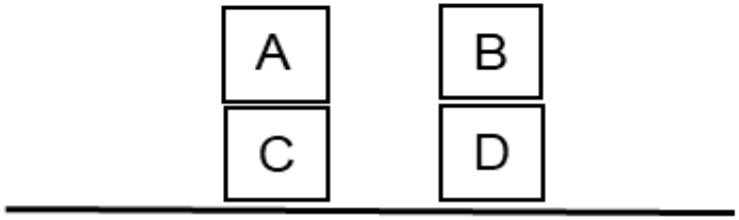
Start

onTable(A), onTable(B), onTable(C),
on(D,A), on(E,C)
clear(D), clear(B), clear(E), AE



Goal

on(B,D), on(A,C)



Q2. Let State Space Planning find a shortest plan. Which of the following can be the first action of the shortest plan?

a) Putdown(C)

b) Pickup(B)

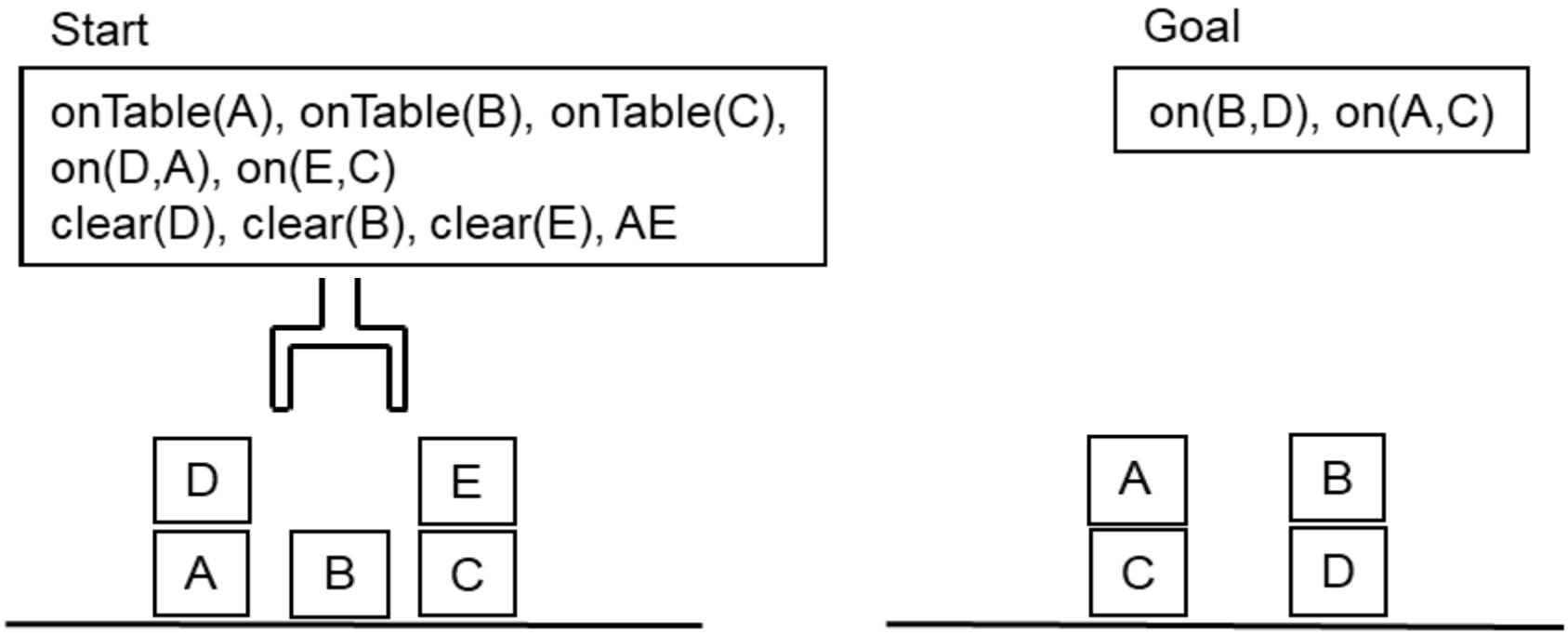
c) Pickup(E)

d) Unstack(D,A)

e) Unstack(B,D)

f) Unstack(E,C)

Ans : d & f



Q4. How many actions are there in the shortest plan to achieve the goal?

Answer: 8

Possible (shortest) plans:

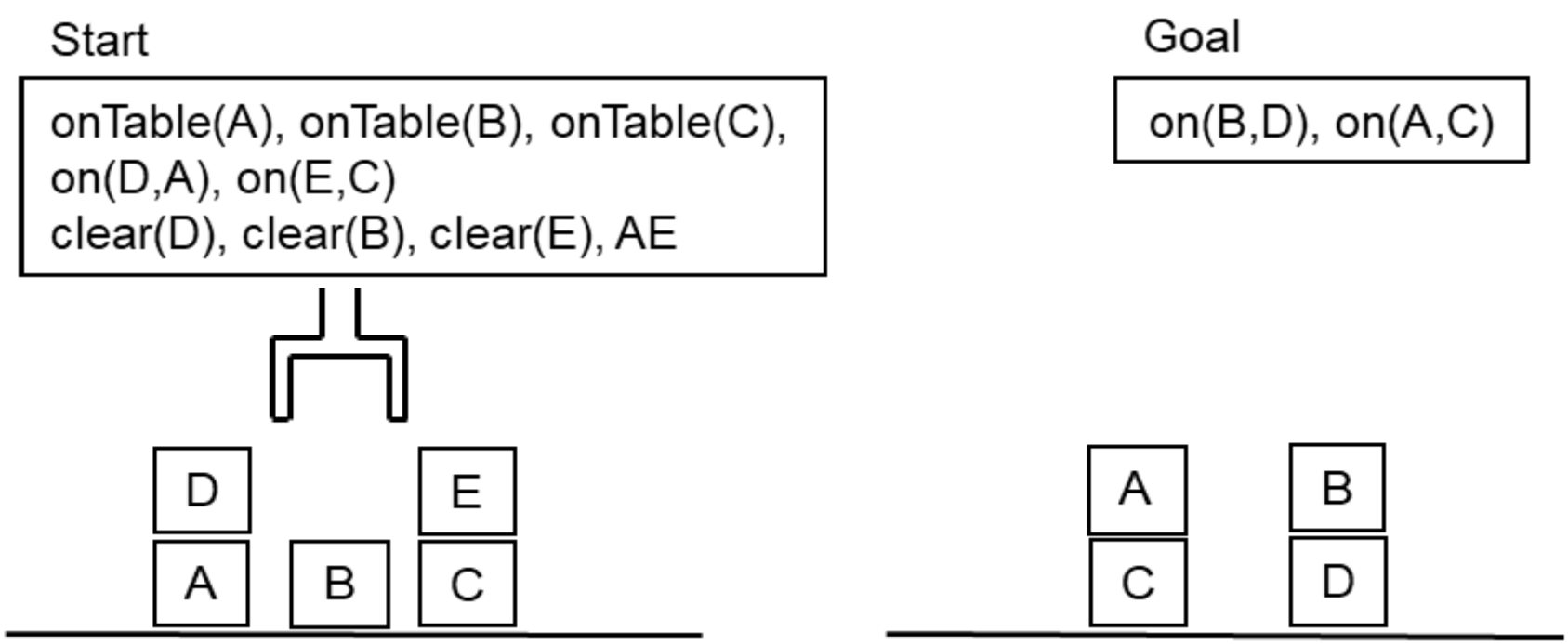
Unstack(D,A), Putdown(D), Pickup(B), Stack(B,D), Unstack(E,C), Putdown(E), Pickup(A), Stack(A,C)

or

Unstack(E,C), Putdown(E), Unstack(D,A), Putdown(D), Pickup(B), Stack(B,D), Pickup(A), Stack(A,C)

or

Unstack(E,C), Putdown(E), Unstack(D,A), Putdown(D), Pickup(A), Stack(A,C), , Pickup(B), Stack(B,D)



14. Which of the following are relevant actions for the above problem?

a) Putdown(D,A)

b) Pickup(B)

c) Stack (A,C)

d) Unstack(D,A)

e) Stack(B,D)

f) Unstack(E,C)

Hint : relevant actions are those which make at least one of the goal predicates true

Ans: c & e

Partial order Planning

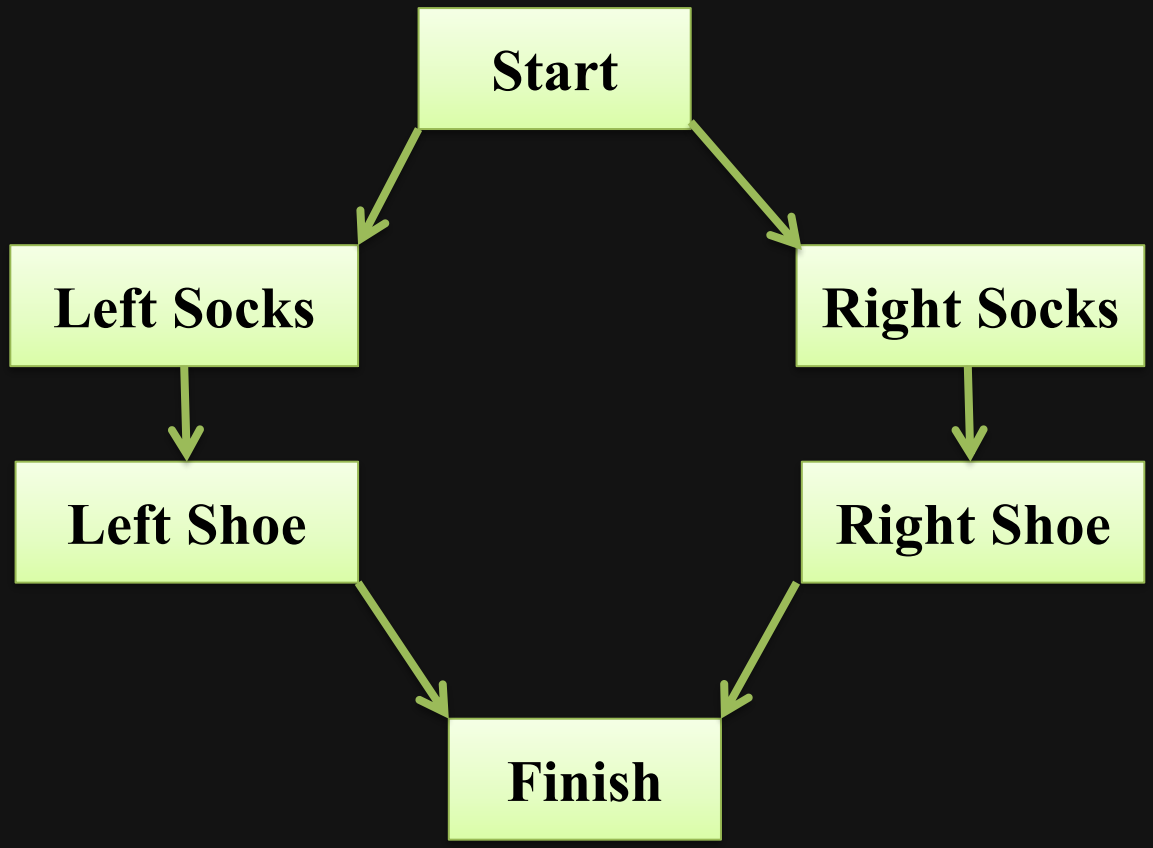
- In case of partial order planning (POP), the ordering of the actions is partial
- Partial order planning does not specify which action will come first out of the two actions, which are placed in plan
- With partial order planning, problem can be decomposed, so it can work well in case of non-cooperative environment
- E.g. wearing shoe

Partial order Planning

- A partial order planning combines two action sequences
- Once these actions are taken we achieve our goal and reach the final state
- Every plan has four main components, which can be given as
  1. Set of actions
  2. Set of ordering constraints/ preconditions
  3. Set of casual links
  4. Set of open preconditions
- consistent plan is a solution for POP problems

Partial order Planning

- E.g. wearing shoe



Conditional Planning

- Conditional planning has to work regardless of the outcome of an action
- Conditional planning can take place in fully observable environment
- The outcome of actions can not be determined, so the environment is said to be non-deterministic
- In conditional planning one can check what is happening in the environment at predetermined points of the plan to deal with ambiguous actions
- Conditional planning can also take place in the partially observable environment, where we can not keep a track on every state

Conditional Planning

- It's a way to deal with uncertainty by checking what is actually happening in the environment at predetermined points in the plan. (Conditional Steps)

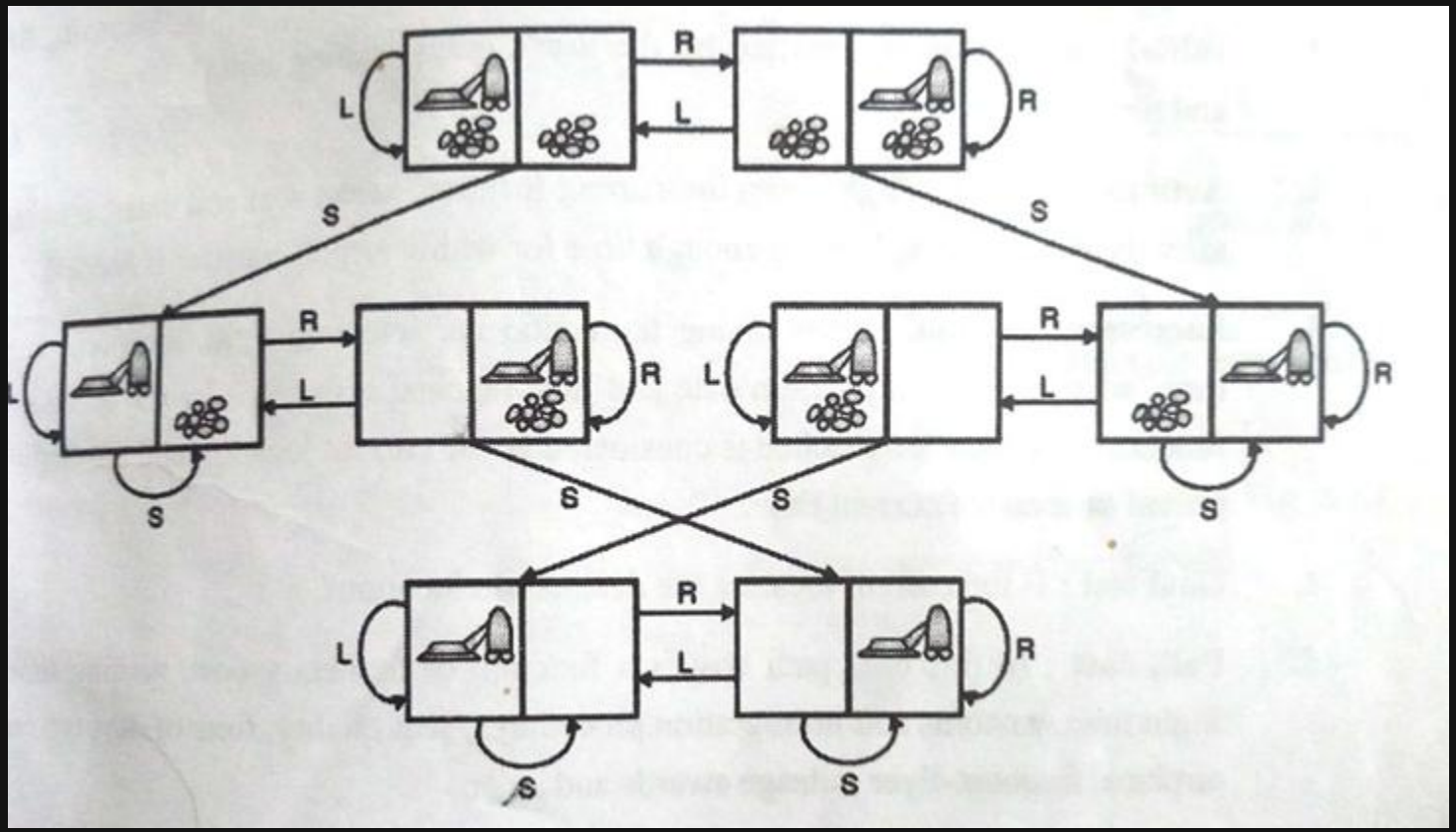
Example:

- Check whether Delhi airport is operational. If so, fly there; otherwise, fly to Mumbai airport.

Conditional Planning

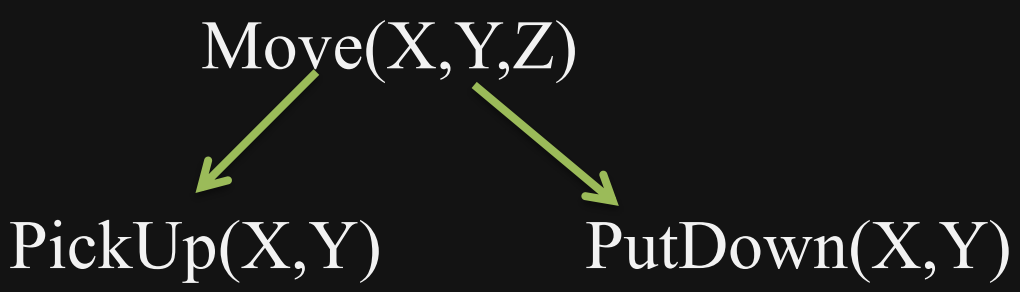
Example 2:

- The vacuum world example



Hierarchical Planning

- Hierarchical planning is also called as plan decomposition.
- Generally plans are organized in a hierarchical format
- Complex actions can be decomposed into more primitive actions and it can be denoted with the help of links between various states at different levels of the hierarchy (This is called as operator expansion)
- E.g.



Hierarchical Planning

Hierarchy of actions:

- In terms of major and minor actions, hierarchy of action can be decided
- Minor activities would cover more precise activities to accomplish the major activities

Planner:

- First identify a hierarchy of major conditions
- Construct a plan in levels, so we postpone the details to next level
- Patch major levels as details actions become visible
- Finally demonstrate